

Relatório Técnico Algoritmo Minimax

Integrantes do grupo:

GABRIELLA NASCIMENTO DOS SANTOS DA SILVA
JOÃO VINÍCIUS DE ALMEIDA ARGOLO
JOSÉ VICTOR RIBEIRO DE JESUS
MARIANA SILVA COSTA
TIAGO RODRIGUES DOS SANTOS

1. Descrição do Problema

O problema consiste em implementar um agente inteligente capaz de jogar Damas de forma competitiva contra um ser humano. O jogo de Damas é classificado como um jogo de informação perfeita, soma zero e turnos alternados. O desafio para a IA é explorar o vasto espaço de estados (possíveis configurações do tabuleiro) para selecionar a jogada que maximiza suas chances de vitória, enquanto antecipa as respostas ótimas do adversário.

2. O Algoritmo de IA: Minimax com Poda Alfa-Beta

O algoritmo escolhido para a solução é o **Minimax com Poda Alfa-Beta**. Ele se encaixa perfeitamente pois:

- **Minimax**: Modela o jogo como uma árvore de decisão onde a IA (MAX) tenta maximizar o valor da utilidade e o humano (MIN) tenta minimizá-lo.
- **Poda Alfa-Beta**: Otimiza a busca ao "podar" (ignorar) ramos da árvore que comprovadamente não afetarão a decisão final, permitindo que a IA explore profundidades maiores (neste projeto, **depth=6**) em menos tempo.

3. Correspondência entre Pseudocódigo e Implementação

Abaixo, detalhamos como o pseudocódigo da imagem fornecida foi "reconhecido" e implementado no arquivo de lógica da IA.

A. Função Principal: ALPHA-BETA-SEARCH

No pseudocódigo, a função inicia a busca definindo o jogador e chamando **MAX-VALUE** com os limites $-\infty$ e $+\infty$.

No Código:

```
def ALPHA_BETA_SEARCH(game, state, depth=6):  
    player = game.TO_MOVE(state)  
    value, move = MAX_VALUE(game, state, -math.inf, math.inf, player, depth)  
    return move
```

- **Correspondência:** A implementação segue fielmente a assinatura e a lógica de inicialização dos parâmetros de poda.

B. Função MAX-VALUE

O pseudocódigo define que, se o estado for terminal, retorna a utilidade. Caso contrário, itera sobre as ações, atualiza α e verifica o corte se $v \geq \beta$.

No Código:

```
def MAX_VALUE(game, state, alpha, beta, player, depth):
    if game.IS_TERMINAL(state) or depth == 0:
        return game.UTILITY(state, player), None

    v = -math.inf
    best_move = None

    for action in game.ACTIONS(state):
        v2, _ = MIN_VALUE(game, game.RESULT(state, action),
                           alpha, beta, player, depth-1)
        if v2 > v:
            v = v2
            best_move = action
            alpha = max(alpha, v)

        if v >= beta:
            return v, best_move

    return v, best_move
```

- **Correspondência:** A estrutura for each a in $\text{game.ACTIONS}(\text{state})$ e a lógica de atualização de α são idênticas ao proposto no material teórico.

C. Função MIN-VALUE

Similarmente, MIN-VALUE busca o menor valor possível, atualizando β e realizando o corte se $v \leq \alpha$.

No Código:

```
def MIN_VALUE(game, state, alpha, beta, player, depth):
    if game.IS_TERMINAL(state) or depth == 0:
        return game.UTILITY(state, player), None

    v = math.inf
    best_move = None

    for action in game.ACTIONS(state):
        # Simula a jogada e chama MAX novamente
        v2, _ = MAX_VALUE(game, game.RESULT(state, action),
                           alpha, beta, player, depth-1)
        if v2 < v:      # Se encontramos valor menor, atualizamos
            v = v2
            best_move = action
            # Atualiza beta (melhor valor garantido para MIN)
            beta = min(beta, v)

        # Se o valor atual já é menor ou igual ao alpha,
        # MAX nunca deixará esse ramo acontecer,
        # então podemos cortar aqui.
        if v <= alpha:
            return v, best_move

    return v, best_move
```

- **Correspondência:** O código implementa exatamente a lógica de minimização e o teste de interrupção da busca (corte) conforme ilustrado no pseudocódigo.

4. Detalhes Técnicos da Implementação

- **Modelagem do Jogo (CheckersGame):** Contém as funções ACTIONS (movimentos possíveis), RESULT (novo estado após movimento) e UTILITY (pontuação do tabuleiro). A captura é obrigatória conforme as regras implementadas.
- **Interface Gráfica (Streamlit):** Utiliza session_state para manter o estado do tabuleiro entre cliques. O CSS personalizado renderiza o tabuleiro e as peças (incluindo damas com o ícone ♚).
- **Heurística:** A função UTILITY calcula o valor do estado baseando-se no peso das peças: Peça comum = 1 ponto, Dama = 3 pontos.